

COMSATS University Islamabad
Attock Campus



Department Of Computer Science

Course	Information Security
Instructor	Ma'am Ambreen Gul
Assignment No.	03

Student Details

Registration No.	Name
FA24-BSE-033	Zineera Zainab Qureshi

Task 1: Basic Blockchain Implementation (Python)

CODE:

```
1 import hashlib
2 import datetime
3
4 # Block class
5 class Block:
6     def __init__(self, index, timestamp, data, previous_hash):
7         self.index = index
8         self.timestamp = timestamp
9         self.data = data
10        self.previous_hash = previous_hash
11        self.hash = self.calculate_hash()
12
13    # Function to calculate hash using SHA-256
14    def calculate_hash(self): 2 usages
15        block_string = str(self.index) + str(self.timestamp) + str(self.data) + str(self.previous_hash)
16        return hashlib.sha256(block_string.encode()).hexdigest()
17
18
19 # Blockchain class
20 class Blockchain: 1 usage
21     def __init__(self):
22         self.chain = [self.create_genesis_block()]
23
24    # First block (Genesis Block)
25    def create_genesis_block(self): 1 usage
26        return Block(index=0, datetime.datetime.now(), data="Genesis Block", previous_hash="0")
27
28    # Get last block
29    def get_latest_block(self): 1 usage
30        return self.chain[-1]
31
32    # Add new block
33    def add_block(self, data): 5 usages (1 dynamic)
34        previous_block = self.get_latest_block()
35        new_block = Block(len(self.chain), datetime.datetime.now(), data, previous_block.hash)
36        self.chain.append(new_block)
37
38    # Display blockchain
39    def display_chain(self): 2 usages
40        for block in self.chain:
41            print("Index:", block.index)
42            print("Timestamp:", block.timestamp)
43            print("Data:", block.data)
44            print("Previous Hash:", block.previous_hash)
45            print("Hash:", block.hash)
46        print("-" * 40)
47
48    # Create blockchain
49    my_blockchain = Blockchain()
```

```
50 # Add blocks
51 my_blockchain.add_block("Block 1 Data")
52 my_blockchain.add_block("Block 2 Data")
53 my_blockchain.add_block("Block 3 Data")
54 my_blockchain.add_block("Block 4 Data")
55
56 # Display chain
57 print("Original Blockchain:")
58 my_blockchain.display_chain()
59
60 # Modify a block to show effect
61 print("\nAfter Modifying Block 2 Data:\n")
62 my_blockchain.chain[2].data = "Hacked Data"
63 my_blockchain.chain[2].hash = my_blockchain.chain[2].calculate_hash()
64
65 my_blockchain.display_chain()
```

OUTPUT:

Original Blockchain:

Index: 0

Timestamp: 2026-05-01 22:15:39.886485

Data: Genesis Block

Previous Hash: 0

Hash: dea9e6787762dfc2979ffb48f135fe14abf74e697dc5397fbfac10f6447eb7fd

Index: 1

Timestamp: 2026-05-01 22:15:39.887459

Data: Block 1 Data

Previous Hash: dea9e6787762dfc2979ffb48f135fe14abf74e697dc5397fbfac10f6447eb7fd

Hash: e6ff02b895f7b9001e197d79fa6ce43a8db8b085624e58032b9d3c7823996711

Index: 2

Timestamp: 2026-05-01 22:15:39.887480

Data: Block 2 Data

Previous Hash: e6ff02b895f7b9001e197d79fa6ce43a8db8b085624e58032b9d3c7823996711

Hash: aa0936582e5d70111c6afc99840565b047723122aebc2f34f3c875a13c9e33c6

Index: 3

Timestamp: 2026-05-01 22:15:39.887493

Data: Block 3 Data

Previous Hash: aa0936582e5d70111c6afc99840565b047723122aebc2f34f3c875a13c9e33c6
Hash: 25a6ec84dfcafd7ea7e7949c80ce229d4ec848d3e1fabddaa9e94b62d2d2b5a6

Index: 4

Timestamp: 2026-05-01 22:15:39.887498

Data: Block 4 Data

Previous Hash: 25a6ec84dfcafd7ea7e7949c80ce229d4ec848d3e1fabddaa9e94b62d2d2b5a6

Hash: 55ebe6cfb0176d10d964fd3f3052ea64e7fbc5638bd01d21b75a04a1c9df101a

After Modifying Block 2 Data:

Index: 0

Timestamp: 2026-05-01 22:15:39.886485

Data: Genesis Block

Previous Hash: 0

Hash: dea9e6787762dfc2979ffb48f135fe14abf74e697dc5397fbfac10f6447eb7fd

Index: 1

Timestamp: 2026-05-01 22:15:39.887459

Data: Block 1 Data

Previous Hash: dea9e6787762dfc2979ffb48f135fe14abf74e697dc5397fbfac10f6447eb7fd

Hash: e6ff02b895f7b9001e197d79fa6ce43a8db8b085624e58032b9d3c7823996711

Index: 2

Timestamp: 2026-05-01 22:15:39.887480

Data: Hacked Data

Previous Hash: e6ff02b895f7b9001e197d79fa6ce43a8db8b085624e58032b9d3c7823996711

Hash: 5bbbf02b0f027260cf8397ecfc3e150f8909bd729c4b1a48d55f0e9537d296de

Index: 3

Timestamp: 2026-05-01 22:15:39.887493

Data: Block 3 Data

Previous Hash: aa0936582e5d70111c6afc99840565b047723122aebc2f34f3c875a13c9e33c6

Hash: 25a6ec84dfcafd7ea7e7949c80ce229d4ec848d3e1fabddaa9e94b62d2d2b5a6

Index: 4

Timestamp: 2026-05-01 22:15:39.887498

Data: Block 4 Data

Previous Hash: 25a6ec84dfcafd7ea7e7949c80ce229d4ec848d3e1fabddaa9e94b62d2d2b5a6

Hash: 55ebe6cfb0176d10d964fd3f3052ea64e7fbc5638bd01d21b75a04a1c9df101a

Process finished with exit code 0

Explanation:

- Each block contains:.
- Index → position of block
- Timestamp → time of creation
- Data → actual information.
- Previous Hash → link to previous block

Line with ord/chr:

Converts letter → number → shifts → back to letter.

Line: `def caesar_decrypt(ciphertext, shift):`

- Defines the decryption function.
- ciphertext is the encrypted message.
- shift is the same shift used in encryption.
- ISHA-256 is used to generate hash

Task 2: ECC (Elliptic Curve Cryptography - Basic)

CODE:

```
1  # Elliptic Curve parameters:  $y^2 = x^3 + ax + b \pmod p$ 
2  a = 2
3  b = 3
4  p = 97 # small prime
5
6  # ----- Point Operations -----
7
8  def point_add(P, Q): 3 usages
9      if P is None:
10         return Q
11      if Q is None:
12         return P
13
14      x1, y1 = P
15      x2, y2 = Q
16
17      # Point at infinity case
18      if x1 == x2 and (y1 + y2) % p == 0:
19         return None
20
21      # Point doubling
22      if P == Q:
23         return point_double(P)
```



```

25     # Slope
26     m = ((y2 - y1) * pow(x2 - x1, -1, p)) % p
27
28     x3 = (m*m - x1 - x2) % p
29     y3 = (m*(x1 - x3) - y1) % p
30
31     return (x3, y3)
32
33
34 def point_double(P): 2 usages
35     if P is None:
36         return None
37
38     x, y = P
39
40     m = ((3*x*x + a) * pow(2*y, -1, p)) % p
41
42     x3 = (m*m - 2*x) % p
43     y3 = (m*(x - x3) - y) % p
44
45     return (x3, y3)
46
47 # scalar multiplication (k * P)
48
49 def scalar_mult(k, P): 5 usages
50     result = None
51     temp = P
52
53     while k > 0:
54         if k % 2 == 1:
55             result = point_add(result, temp)
56             temp = point_double(temp)
57             k = k // 2
58
59     return result
60
61
62 # ----- Key Generation -----
63
64 G = (3, 6)      # Base point
65 d = 2           # Private key (small value)
66
67 Q = scalar_mult(d, G) # Public key
68
69 print("Private Key:", d)
70 print("Public Key:", Q)
71
72

```

```

73 # ----- Message (Small Value Test) -----
74 message = 5 # small numeric message
75
76 # Convert message to point
77 M = scalar_mult(message, G)
78 print("Message Point:", M)
79
80
81 # ----- Encryption -----
82 k = 3 # random small number
83
84 C1 = scalar_mult(k, G)
85 C2 = point_add(M, scalar_mult(k, Q))
86
87 print("\nEncrypted:")
88 print("C1:", C1)
89 print("C2:", C2)
90
91
92 # ----- Decryption -----
93 def point_neg(P): 1 usage
94     if P is None:
95         return None
96     x, y = P
97     return (x, (-y) % p)
98
99 dC1 = scalar_mult(d, C1)
100
101 M_decrypted = point_add(C2, point_neg(dC1))
102
103 print("\nDecrypted Message Point:", M_decrypted)

```

OUTPUT:

```

↓ Private Key: 2
⇌ Public Key: (80, 10)
⇌ Message Point: None
🖨 Encrypted:
🗑 C1: (80, 87)
C2: (3, 6)

Decrypted Message Point: None

Process finished with exit code 0
|

```


Explanation:

- ECC is based on equation:
$$y^2 = x^3 + ax + b$$
- Operations:
- Point Addition → adding two points
- Point Doubling → doubling same point
- Private Key → secret number
- Public Key → generated using base point

Used in real systems because it provides **high security with small key size**

Security Analysis:

- Blockchain ensures:
- Data cannot be changed easily ● Each block depends on previous block ● ECC ensures:
- Strong encryption
- Efficient key generation ● Both are widely used in:
 - Cryptocurrency
- Secure communication